■ COMPREHENSIVE PASSWORD RESET BUG GUIDE

Step-by-Step Reproduction, New Vulnerabilities & Exploitation Techniques

---

Covers: CVE-2026-23760 | CVE-2026-26273 | CVE-2026-41940 | OTP Bypass | Token Theft | Account Takeover

Edition: May 2026 | **Category:** Authentication Bypass | **Severity:** Critical

■■ DISCLAIMER: This guide is for educational and authorized security testing purposes only. Always obtain proper permission before testing any system.

■ TABLE OF CONTENTS

1. INTRODUCTION TO PASSWORD RESET VULNERABILITIES

Password reset functionality is the **backdoor to your authentication system**. Its job is to let users back in without their password, which means it must bypass the normal credential check by design. That makes it structurally weaker than login, and attackers exploit that gap directly. Most teams add strong bot protection to login but neglect the reset endpoint entirely. Reset flows reveal useful information to attackers: whether an account exists, which email addresses are registered, and how quickly reset tokens expire. Reset forms are public-facing and easy to automate — a bot can trigger thousands of reset requests per hour against a form with no rate limiting or CAPTCHA.

■ **Four Major Abuse Patterns:**

Pattern	What Attackers Do	What They Gain
Account Enumeration	Submit reset requests for many emails and observe response differences	Validated list of real account emails for phishing or credential stuffing
Reset Flooding	Trigger large numbers of reset emails automatically	Inbox overload, harassment, or mail infrastructure degradation
Token Theft	Intercept reset links via referrer headers, archived URLs, or brute-force short tokens	Valid reset token for account takeover without email access
Weak Recovery Design	Exploit reusable tokens, non-expiring links, or guessable security questions	Account access through recovery path without breaking login

2. CVE-2026-23760 — SMARTERMAIL AUTHENTICATION BYPASS

■ Severity: Critical (CVSS 9.8) | Status: Active Exploitation in the Wild | CISA KEV: Listed

A critical authentication bypass in SmarterMail email server software. The **force-reset-password** API endpoint permits unauthenticated requests and fails to verify existing passwords or reset tokens when resetting system administrator accounts. An attacker can reset ANY user's password (including admin) with a single HTTP POST request.

BUG #1: SmarterMail Admin Password Reset Bypass

Target: SmarterMail Web API (ports 9998/17017)

Impact: Full System Compromise → RCE as SYSTEM/root

■ ANSWER: Authentication Bypass via force-reset-password API

Root Cause: The ForceResetPassword method has two code paths based on the user-controllable **IsSysAdmin** Boolean parameter. When IsSysAdmin=true, the application completely bypasses OldPassword validation — despite accepting it in the request.

■ Key Vulnerability: The application trusts client-supplied data to determine whether to enforce security controls. Regular users get password validation; administrators do not.

■ Steps to Reproduce:

- **■ Step 1 — Reconnaissance:** Identify SmarterMail instance via OSINT (html:'SmarterMail') or port scan (9998/17017).
- **■ Step 2 — Verify Vulnerability:** Check version fingerprinting. Versions < Build 9511 are vulnerable.
- **■ Step 3 — Exploit Delivery:** Send crafted POST to /api/v1/auth/force-reset-password
- **■ Step 4 — Admin Login:** Use newly set credentials to access admin panel.
- **■■ Step 5 — RCE:** Navigate to Settings → Volume Mounts → inject OS command in 'Volume Mount Command' field.

■ Proof of Concept (PoC) Request:

```
POST /api/v1/auth/force-reset-password HTTP/1.1 Host: target-smartermail-server:9998
Content-Type: application/json { "IsSysAdmin": "true", "OldPassword":
"irrelevant-not-checked", "Username": "admin", "NewPassword":
"AttackerPassword123!@#", "ConfirmPassword": "AttackerPassword123!@#" }
```

■ Expected Response:

```
HTTP/1.1 200 OK { "success": true, "resultCode": 200 }
```

■ Key Takeaways:

- Global Impact: ~10,000+ vulnerable instances worldwide.
- Active Exploitation: Confirmed since January 17, 2026 (2 days post-patch).
- Patch Diffing Attack: Attackers reverse-engineered vulnerability from patch differences.
- RCE Capability: Admin access includes built-in OS command execution via Volume Mounts.

3. CVE-2026-26273 — KNOWN PLATFORM TOKEN EXPOSURE

■ Severity: Critical | Type: CWE-200 (Information Disclosure) | Affected: Known < v1.6.3

The Known social publishing platform exposes password reset tokens directly within **hidden HTML input fields** on the password reset page. This allows unauthenticated attackers to retrieve reset tokens for ANY user account by simply initiating a password reset request — completely bypassing email-based authentication controls.

BUG #2: Known Platform Reset Token Leakage

Target: Known social publishing platform (< v1.6.3)

Impact: Full Account Takeover without email access

■ ANSWER: Reset Token Exposed in Hidden HTML Input Field

Root Cause: The password reset handler generates a token and incorrectly includes it in the HTML response body as a hidden form field value. Instead of storing the token server-side and only referencing it via secure email link, the application renders it in the page source.

■ Key Flaw: This violates the fundamental principle that password reset tokens should ONLY be transmitted through secure, out-of-band channels such as email.

■ Steps to Reproduce:

- ■ **Step 1:** Navigate to the target Known installation's password reset page.
- ✉■ **Step 2:** Enter the victim's email address to initiate a password reset.
- ■■ **Step 3:** View the page source (Ctrl+U) to locate the hidden input field containing the reset token.
- ■ **Step 4:** Extract the token value from the HTML source.
- ■ **Step 5:** Use the extracted token directly to reset the victim's password and gain full account access.

■ Detection Indicators:

- Multiple password reset requests from same IP targeting different accounts.
- Successful password resets without corresponding email link clicks in logs.
- Account access from new IP addresses immediately following reset requests.
- Unusual patterns of reset page access followed by immediate login attempts.

■■ **Mitigation:** Upgrade to Known v1.6.3+ | Audit reset logs | Force password resets for potentially compromised accounts | Review active sessions

4. CVE-2026-41940 — CPANEL/WHM AUTHENTICATION BYPASS

■ Severity: CVSS 9.8 (Critical) | Impact: ~1.5M exposed servers | Exploitation: In-the-wild since Feb 2026

A pre-authentication remote auth bypass in cPanel & WHM that chains a **CRLF injection** in the session writer with an **encryption-skip** triggered by a malformed cookie. It uses a quirk in how cPanel caches sessions to 'promote' the injection into a privileged login. CISA added it to the KEV catalog.

BUG #3: cPanel/WHM Pre-Auth Admin Takeover

Target: cPanel & WHM (all supported versions after 11.40)

Impact: Full server compromise + all hosted sites

■ ANSWER: CRLF Injection + Encryption-Skip + Session Cache Poisoning Chain

Attack Chain: (1) CRLF injection in session writer → (2) Malformed cookie triggers encryption skip → (3) Session cache 'promotes' unauth session to privileged login. Two representations of the same data (raw vs. cache) create a parser disagreement that is exploitable.

■ Key Lesson: 'Already authenticated' flags need cryptographic binding. A boolean in a session that says 'skip the password check' must be unforgeable, not just present.

■ Steps to Reproduce (Conceptual):

- **■ Step 1:** Identify cPanel instance (ports 2083/2087/2095/2096).
- **■ Step 2:** Craft HTTP request with CRLF injection in session parameter.
- **■ Step 3:** Supply malformed cookie to trigger encryption-skip condition.
- **■ Step 4:** Session writer processes injection, creating poisoned session file.
- **■ Step 5:** Access session via cache path — parser reads 'authenticated=true' flag.
- **■ Step 6:** Gain WHM root-level access without credentials.

■■ Mitigation Checklist:

- Patch: Update via /scripts/upcp --force | Verify with /usr/local/cpanel/cpanel -V

- Rotate: Force-reset root and all WHM reseller passwords | Rotate API tokens | Re-issue SSH keys
- Purge: Clear sessions in /var/cpanel/sessions/raw/ and /var/cpanel/sessions/cache/
- Hunt: Audit cron entries, ~/.ssh/authorized_keys, custom WHM hooks
- Contain: Block inbound TCP/2083, 2087, 2095, 2096 at perimeter if patching delayed

5. PASSWORD RESET POISONING TECHNIQUES

Password Reset Poisoning occurs when an attacker manipulates how the reset link is generated, causing the reset email to contain a link pointing to an attacker-controlled domain. When the victim clicks the link, the token is sent to the attacker's server.

BUG #4: Host Header Manipulation (Reset Poisoning)

Target: Applications using Host header to generate reset links

Impact: Token interception via attacker-controlled domain

■ ANSWER: Manipulate Host Header or X-Forwarded-Host to Redirect Reset Link

Technique: The application uses the Host header (or X-Forwarded-Host) to construct the password reset URL in the email. By changing this header, the attacker can make the reset link point to their domain.

■ **Key Insight:** When victim clicks the poisoned link, the reset token is leaked to the attacker's server via URL parameters or referrer header.

■ Steps to Reproduce:

- ■ **Step 1:** Intercept the password reset request in Burp Suite.
- ■ **Step 2:** Modify the Host header to attacker.com or add X-Forwarded-Host: attacker.com
- ■ **Step 3:** Submit the request and check the reset email received.
- ■ **Step 4:** Verify if the reset link points to attacker.com instead of the legitimate domain.
- ■ **Step 5:** Set up a listener on attacker.com to capture the token when victim clicks.

■ PoC — CRLF Injection Variant:

```
GET /resetpassword?%0d%0aHost:%20attacker.com HTTP/1.1
```

■ PoC — URL Manipulation:

```
POST https://attacker.com/resetpassword.php HTTP/1.1 POST
@attacker.com/resetpassword.php HTTP/1.1 POST :@attacker.com/resetpassword.php
HTTP/1.1 POST /resetpassword.php@attacker.com HTTP/1.1
```

6. OTP & 2FA BYPASS METHODS

One-Time Passwords (OTPs) are widely used in MFA systems, but weak implementations create vulnerable entry points. Attackers are no longer trying to guess passwords — they are stealing entire authenticated sessions in real time using Adversary-in-the-Middle (AiTM) attacks and OTP bots.

BUG #5: OTP Brute Force & Predictable Tokens

Target: Applications with short/predictable OTP codes

Impact: Account takeover via automated OTP guessing

■ ANSWER: Brute-Force Short OTPs or Reverse-Engineer Predictable Generation Algorithms

Technique 1 — Brute Force: If the OTP is 4-6 digits with no rate limiting, an attacker can automate attempts. With 1,000,000 combinations for 6 digits, a fast bot can test thousands per minute.

Technique 2 — Predictable Generators: Weak OTP generation algorithms produce easily predictable tokens. Attackers reverse-engineer the algorithm to craft valid OTPs without brute force.

Technique 3 — Session Extension: If session timeouts are not enforced, an OTP's validity can be improperly extended, giving attackers more time to exploit stolen credentials.

■ Steps to Reproduce OTP Brute Force:

- ■ **Step 1:** Initiate password reset and request OTP to target email/phone.
- ■■ **Step 2:** Note the OTP length (4, 6, or 8 digits) and expiration time.
- ■ **Step 3:** Use Burp Intruder or custom script to automate OTP submission.
- ■ **Step 4:** Observe response differences for valid vs. invalid OTPs.
- ■ **Step 5:** Upon valid OTP, proceed to password reset and take over account.

■ NEW BUG: MFA Downgrade Attack (2026)

Most organizations that deploy passkeys or security keys still maintain fallback authentication methods: SMS codes, authenticator apps, or password reset flows. Attackers exploit this by manipulating the login flow to force the system to offer a less secure method. If a user can reset their passkey by answering security questions or receiving an SMS code, the entire phishing-resistant chain is only as strong as its weakest fallback.

■ Steps to Reproduce MFA Downgrade:

- ■ **Step 1:** Attempt login with valid credentials on an account with FIDO2/passkey.
- ■ **Step 2:** When prompted for security key, look for 'Use another method' or 'Try another way' option.
- ■ **Step 3:** Select SMS or security questions as fallback.
- ■ **Step 4:** Complete the weaker authentication method.
- ■ **Step 5:** Gain access without the phishing-resistant factor.

7. TOKEN THEFT & BRUTE FORCE ATTACKS

Password reset tokens are the keys to the kingdom. If they are short, predictable, leaked in responses, or reusable, attackers can bypass the entire email verification step.

BUG #6: Token Leakage via Referrer Header

Target: Reset pages containing external links

Impact: Token leaked to third-party domains via Referrer

■ ANSWER: Click External Links on Reset Page to Leak Token via Referrer

Technique: Open the password reset link and click on any external links available on the page (social media buttons, help links, etc.). The browser sends the full URL (including the reset token) in the Referrer header to the external domain.

■ **Key Insight:** If the external domain is attacker-controlled or logs are accessible, the token is compromised.

■ Steps to Reproduce:

- ■ **Step 1:** Request a password reset for your own test account.
- ■ **Step 2:** Open the reset link from the email.
- ■ **Step 3:** Inspect the reset page for any external links (Facebook, Twitter, Help, etc.).
- ■■ **Step 4:** Click on an external link and intercept the request.
- ■■ **Step 5:** Check the Referrer header — it contains the full reset URL with token.

■ NEW BUG: Token Leakage in JavaScript/Response (2026)

Modern SPAs (Single Page Applications) sometimes include the reset token in JavaScript variables or API responses for 'convenience.' An attacker can find these tokens by inspecting page source, network responses, or JavaScript files loaded by the reset page.

■ Steps to Reproduce JS Token Leakage:

- ■ **Step 1:** Initiate password reset and load the reset page.
- ■ **Step 2:** View page source (Ctrl+U) and search for 'token', 'reset', 'key'.
- ■ **Step 3:** Open browser DevTools → Network tab → inspect all XHR/fetch requests.

- ■ **Step 4:** Search JS files loaded by the page for token variables.
- ■ **Step 5:** If token is found in any of these locations, it is exposed to any user with page access.

8. HTTP PARAMETER POLLUTION (HPP)

HTTP Parameter Pollution involves submitting multiple parameters with the same name to confuse the application's parser. Different frameworks handle duplicate parameters differently — some use the first value, some use the last, and some concatenate them. This ambiguity can be exploited in password reset flows.

BUG #7: HPP in Password Reset — Dual Email Injection

Target: Applications parsing duplicate email parameters

Impact: Reset victim's password using attacker's email

■ ANSWER: Inject Two Email Parameters — One for Victim, One for Attacker

Technique: Submit a password reset request with TWO email parameters. The backend parser may process the victim's email for the actual reset but send the confirmation/link to the attacker's email (or vice versa, depending on parser behavior).

■ **Key Insight:** If the application uses the first email for the reset logic and the second for the email notification, the attacker receives the reset link for the victim's account.

■ Steps to Reproduce:

- ■ **Step 1:** Intercept the password reset request in a proxy tool.
- ■ **Step 2:** Modify the request to include duplicate email parameters:
- ■ **Step 3:** Send the modified request and monitor both email inboxes.
- ■ **Step 4:** Check if the reset link is sent to the attacker's email but works for the victim's account.
- ■ **Step 5:** Use the received link to reset the victim's password.

■ PoC Payloads:

```
# Standard form encoding email=victim@gmail.com&email;=attacker@gmail.com #
Array-style (PHP/Ruby) email[]=victim@gmail.com&email;[]=attacker@gmail.com # JSON
object duplication {"email":"victim@gmail.com","email":"attacker@gmail.com"} # JSON
array {"email":["victim@gmail.com","attacker@gmail.com"]} # Pipe/space separated
email=victim@gmail.com|email=attacker@gmail.com
email=victim@gmail.com%20email=attacker@gmail.com
```

9. IDOR IN PASSWORD RESET FLOWS

Insecure Direct Object Reference (IDOR) occurs when an application exposes a reference to an internal implementation object (like a user ID or token) without proper access control. In password reset flows, this can allow an attacker to reset another user's password by manipulating parameters.

BUG #8: IDOR via User ID Parameter in Reset Link

Target: Reset links containing user IDs or predictable tokens

Impact: Reset any user's password by changing the ID parameter

■ ANSWER: Modify User ID or Token Parameter in Reset URL to Target Different Account

Technique: Use paramminer or manual testing to discover additional parameters in reset requests. Append previously known parameters (e.g., uid found while updating profile) to the reset request. If the application uses these parameters without proper validation, you can reset any account.

■ **Key Insight:** Always check if the reset token is tied to a specific user ID. If you can change the user ID while keeping a valid token structure, the application may accept it.

■ Steps to Reproduce:

- ■ **Step 1:** Request a password reset for your own account and capture the reset link.
- ■ **Step 2:** Analyze the reset URL structure. Look for user_id, uid, token, reset_id parameters.
- ■ **Step 3:** Use ParamMiner or ffuf to discover hidden parameters in the reset endpoint.
- ■ **Step 4:** Modify the user_id parameter to target another account (e.g., user_id=1 for admin).
- ■ **Step 5:** Submit the modified request. If successful, you can reset the target account's password.

■ PoC — IDOR with Appended Parameter:

```
# Original reset request POST /api/reset-password HTTP/1.1
{"email":"victim@example.com"} # Modified with IDOR parameter POST
/api/reset-password HTTP/1.1 {"email":"victim@example.com","user_id":"1"} # Or in URL
GET /reset?token=abc123&user;_id=1 HTTP/1.1
```

10. ADVANCED TECHNIQUES & NEW BUG PATTERNS

■ NEW BUG: Registration Endpoint Password Reset ($4,000 Bounty)

A researcher discovered that a registration endpoint (/doRegistrationEntries) was also handling password resets. When an existing email was submitted to the registration API, instead of returning 'email already exists,' it updated the password for that account. This turned a simple convenience endpoint into a critical security hole.

BUG #9: Registration API Reused for Password Reset

Target: Mobile app registration endpoints

Impact: Full account takeover via registration API

■ ANSWER: Backend Logic Reuse — Registration Endpoint Updates Password for Existing Users

Root Cause: The /doRegistrationEntries route was designed for onboarding, but its validation step didn't differentiate between new user creation and existing user password modification. The pseudocode flaw: if user exists, update password instead of throwing error.

■ **Key Lesson:** Separate registration and password logic. Never let one endpoint handle both flows. Enforce uniqueness and block any attempts to reuse registered emails.

■ Steps to Reproduce:

- ■ **Step 1:** Extract .js files from the mobile app APK using apktool.
- ■ **Step 2:** Parse codebase for interesting URLs and build a custom wordlist.
- ■ **Step 3:** Fuzz endpoints with FFUF to find hidden or undocumented paths.
- ■ **Step 4:** Discover /doRegistrationEntries or similar endpoint.
- ■ **Step 5:** Submit existing victim email with new password to registration endpoint.
- ■ **Step 6:** If the API updates the password instead of rejecting, account is compromised.

■ PoC Request:

```
POST /parents/application/v4/admin/doRegistrationEntries HTTP/1.1 Host:
www.target.com Content-Type: application/json { "email": "victim@school.edu",
"password": "NewAttackerPassword123!", "confirmPassword": "NewAttackerPassword123!" }
```

■ NEW BUG: Weak Password Policy Bypass

Some applications accept passwords containing ONLY spaces or null bytes. Try setting the password to ' ' (spaces) or appending null bytes (%00) to bypass complexity checks.

■ PoC:

```
# Space-only password password=%20%20%20%20%20 # Null byte injection
password=validpass%00 # Very long password (DoS) password=A*100000
```

11. DETECTION & MITIGATION STRATEGIES

■■ Defensive Checklist for Password Reset Security:

Control	Implementation	Priority
Consistent Responses	Return the SAME response whether account exists or not. Eliminates enumeration at zero cost.	■ Critical
Rate Limiting	Limit reset requests per IP/email to 3-5 per hour. Blocks flooding and brute force.	■ Critical
CAPTCHA	Add CAPTCHA to reset form. Stops automated abuse but NOT token theft.	■ High
Token Security	Use cryptographically random tokens (256-bit minimum). Expire after 1 hour max. Single-use only.	■ Critical
Out-of-Band Delivery	NEVER embed tokens in HTML/JS/responses. Only send via verified email/SMS.	■ Critical
Host Header Validation	Use a whitelist of allowed domains. Reject requests with manipulated Host headers.	■ High
Session Invalidation	Invalidate all existing sessions and tokens when password is reset. Force re-login.	■ High
2FA Preservation	Do NOT auto-disable 2FA after password reset. Require re-verification of second factor.	■ Critical
Audit Logging	Log all reset requests, token generations, and completions. Alert on anomalies.	■ Medium
Password Complexity	Enforce strong password policy server-side. Reject space-only or null-byte passwords.	■ High

12. BUG BOUNTY REPORTING TEMPLATE

■ **Use this template for consistent, high-quality bug bounty reports:**

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ BUG
BOUNTY REPORT TEMPLATE
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
TITLE: [Critical] Password Reset Vulnerability Leads to Account Takeover on [Target
Name] SEVERITY: Critical (CVSS: 9.8) SUMMARY: The password reset functionality on
[Target Domain] contains a vulnerability that allows an unauthenticated attacker to
reset any user's password without access to their email, leading to complete account
takeover. AFFECTED ENDPOINT: POST https://[target.com]/api/v1/auth/reset-password
STEPS TO REPRODUCE: 1. Navigate to [Target Domain]/forgot-password 2. Enter victim's
email address: victim@example.com 3. Intercept the request and [describe
modification] 4. Observe that [describe vulnerability behavior] 5. Use the
[token/link/response] to reset the password 6. Login with new password:
NewPassword123! PROOF OF CONCEPT: [Attach screenshots/video showing each step]
[Include HTTP request/response logs] IMPACT: - Complete account takeover for any user
- Access to sensitive data, financial info, PII - Potential escalation to
admin/system access - Reputational damage and compliance violations MITIGATION: 1.
[Specific fix recommendation] 2. [Additional hardening steps] 3.
[Detection/monitoring advice] REFERENCES: - OWASP Forgot Password Cheat Sheet -
CWE-200: Exposure of Sensitive Information - [Any relevant CVEs or research]
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

■ **Key Takeaways for Bug Bounty Hunters:**

- Fuzz Every Endpoint — Hidden routes often expose forgotten backend logic.
- Separate Registration and Password Logic — Never let one handle both flows.
- Validate Inputs Server-Side — Don't rely on the client app to enforce logic.
- Re-Test Old Findings — Revisiting apps after updates often reveals new issues.
- Document Everything — Clear reproduction steps lead to faster triage and higher bounties.
- Check for Token Leakage — In HTML, JS, responses, referrer headers, and browser history.
- Test MFA Downgrade — The weakest fallback determines the strength of the entire chain.

■ END OF GUIDE

Generated: May 2026 | For Educational & Authorized Security Testing Only